

Fifty Years of Prolog and Beyond

Published online by Cambridge University Press: 17 May 2022

Integrantes

Laura Menezes

Davi Puddo

Mateus Ottoni

Raquel Motta

Antônio Drumond

Conteúdo

Introdução

Parte I: As primeiras etapas do Prolog

Parte II: Estado Atual do Prolog

Conclusões atuais

Parte III: O futuro do Prolog

Conclusão final

**Fifty Years of
Prolog and beyond**

KÖRNER, P. et al. Fifty Years of Prolog and Beyond. Theory and Practice of Logic Programming, v. 22, n. 6, p. 776-858, 2022. Disponível em:
<https://doi.org/10.1017/S1471068422000102>

This is an Open Access article, distributed under the terms of the Creative Commons Attribution-ShareAlike licence (<http://creativecommons.org/licenses/by/4.0/>), which permits re-use, distribution, and reproduction in any medium, provided the same Creative Commons licence is used to distribute the re-used or adapted article and the original article is properly cited.

doi:10.1017/S1471068422000102 First published online 17 May 2022

*Fifty Years of Prolog and Beyond**

PHILIPP KÖRNER and MICHAEL LEUSCHEL

Institut für Informatik, Universität Düsseldorf, Universitätsstr. 1, D-40225 Düsseldorf, Germany
(e-mails: p.koerner@uni-duesseldorf.de, leuschel@uni-duesseldorf.de)

JOÃO BARBOSA and VÍTOR SANTOS COSTA

Department of Computer Science, Faculty of Science of the University of Porto, Porto, Portugal
(e-mails: joao.barbosa@fc.up.pt, vscoستا@fc.up.pt)

VERÓNICA DAHL

Computing Sciences Department, Simon Fraser University, Burnaby, Canada
(e-mail: veronica_dahl@sfu.ca)

MANUEL V. HERMENEGILDO and JOSE F. MORALES

IMDEA Software Institute and Universidad Politécnica de Madrid (UPM), Madrid, Spain
(e-mails: manuel.hermenegildo@imdea.org, josef.morales@imdea.org)

JAN WIELEMAKER

Centrum voor Wiskunde en Informatica (CWI), Amsterdam, Netherlands
(e-mail: J.Wielemaker@cwi.nl)

DANIEL DIAZ

Centre de Recherche en Informatique, University Paris-1, Paris, France
(e-mail: daniel.diaz@univ-paris1.fr)

SALVADOR ABREU

NOVA-LINCS, University of Évora, Evora, Portugal
(e-mail: spa@uevora.pt)

GIOVANNI CIATTO

Department of Computer Science and Engineering, Alma Mater Studiorum—Università di Bologna, Bologna, Italy
(e-mail: giovanni.ciatto@unibo.it)

Objetivo do trabalho e estrutura

Objetivo: Analisar 50 anos de evolução do Prolog, integrando lições aprendidas e propondo direções futuras.

Estrutura do artigo:

Parte 1: Evolução histórica até o padrão ISO (1995).

Parte 2: Implementações pós-ISO e desafios de compatibilidade.

Parte 3: Análise SWOT + propostas para o futuro.

Definição de Prolog

Características fundamentais:

1. Cláusulas de Horn: Regras lógicas para representar conhecimento.
2. Meta-predicados: Permitem manipular regras como dados.
3. SLD-resolution: Método de busca backtracking (tentativa e erro).
4. Unificação: "Casar" termos com variáveis (ex: $X = 5$).
5. Busca em profundidade: Explora uma opção até o fim antes de voltar.

Teste técnico: "Se um sistema executa este código de forma flexível (união, divisão de listas), é Prolog."

```
append([ ], X, X).  
append([H|X], Y, [H|Z]) :- append(X, Y, Z).
```

Ancestrais e nascimento do Prolog

Antes do Prolog, havia 3 áreas-chave:

IA (LISP) // teorema automático (usando resolução de Robinson) // processamento de linguagem (Q-systems)

Colmerauer queria criar um sistema de tradução automática e, com Kowalski, adaptaram a SL-resolution para análise linguística. Isso levou ao Prolog 0 em 1972.

Marco: introdução revolucionária de corrotinas, que permitiam pausar/adiar a execução de tarefas até que variáveis fossem definidas. → base para Prolog I (1973).

Autores destacam essa colaboração interdisciplinar como crucial para o nascimento do Prolog, mostrando como a ciência avança através de conexões entre áreas.

Early Prolog Systems

Sistemas pioneiros (1972–1995):

- Prolog 0 (1972): Primeira versão, em Fortran, com corrotinas. Lento.
- DEC-10 Prolog (1975): Primeiro compilador eficiente (Edinburgh). Mais rápido, rivalizando com LISP.
- C-Prolog (1982): Portável para sistemas Unix, popularizou a sintaxe padrão.
- MU-Prolog (1982): Introduziu negação segura (evita erros comuns) e módulos.

WAM (1983):

Máquina abstrata que padronizou a compilação de Prolog, tornando-o rápido e portátil.

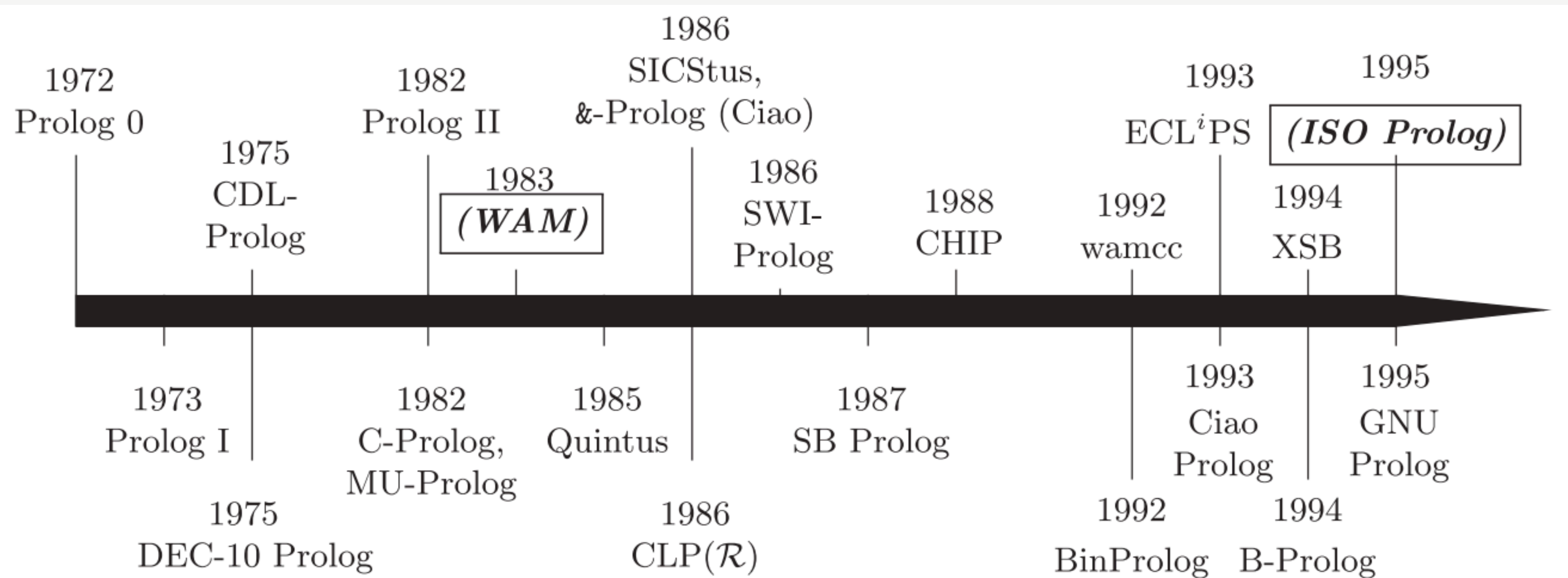


Fig. 1. Approximate timeline of some early Prolog systems (up to the ISO Standard).

*Warren Abstract Machine*₁₉₈₃

Clarificação da implementação em
Prolog do DEC-10.

Uso de compilação em código
intermediário.

Tornou-se a base para
compiladores de Prolog.

Fifth Generation Computer Systems

FGCS

Iniciativa Japonesa 1982

Objetivo

Impulsionar indústria de computação.

Desenvolvimento

Criação de subconjunto de Prolog sem backtracking devido à complexidade.

Resultado

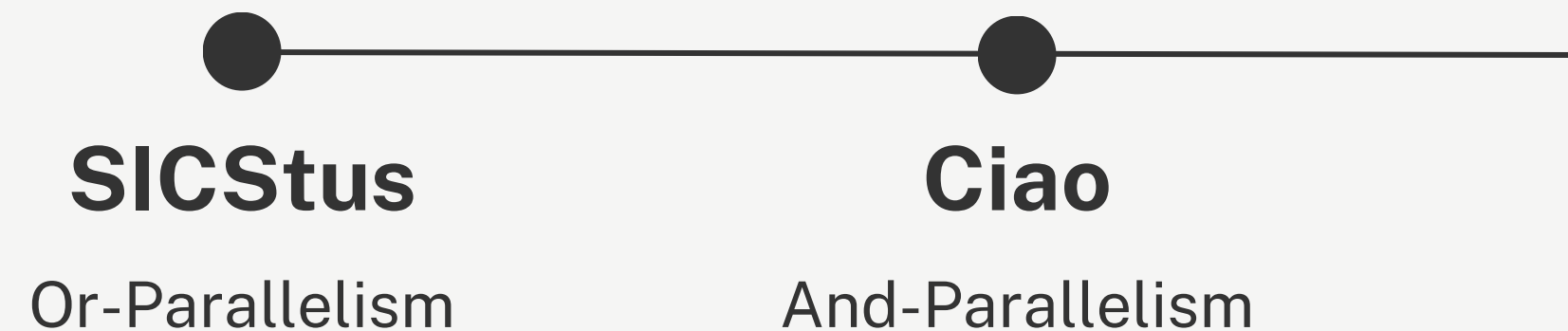
Desenvolvimento da computação paralela.

Paralelismo

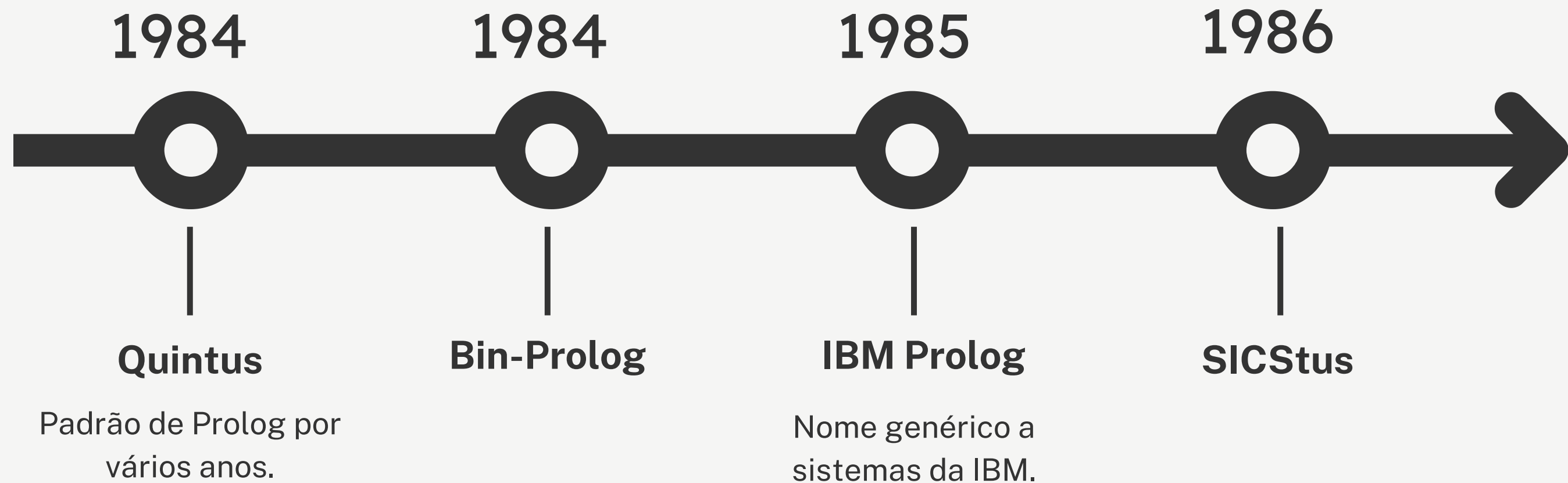
Programação lógica e Prolog reconhecidos como benéficos ao desenvolvimento da execução paralela.

Or-Parallelism: Explora caminhos alternativos paralelamente à execução.

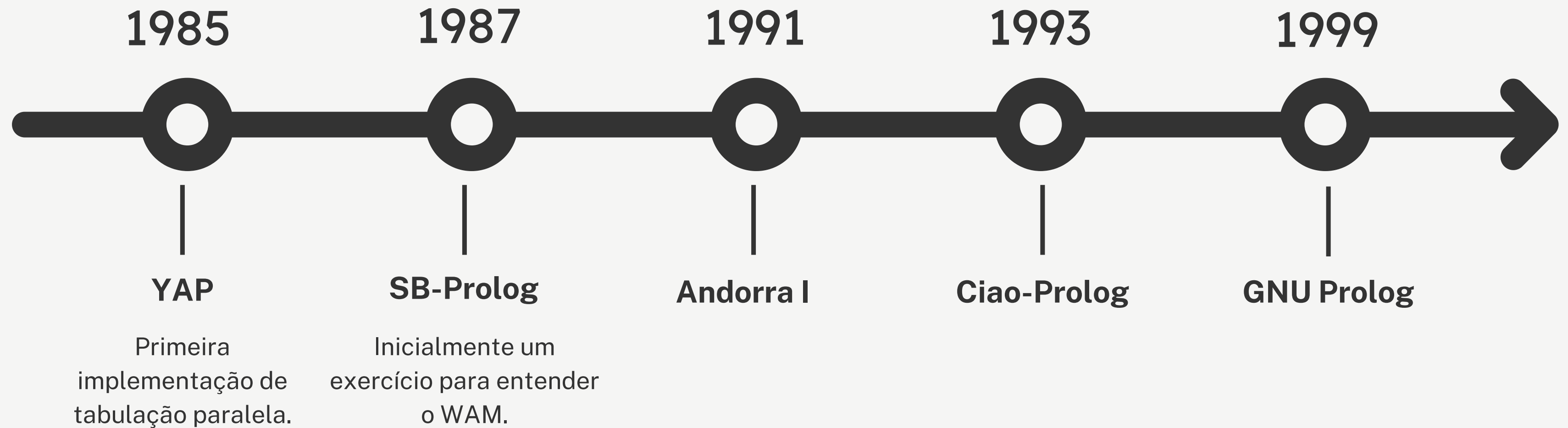
And-Parallelism: O mesmo caminho de execução é executado em passos paralelos



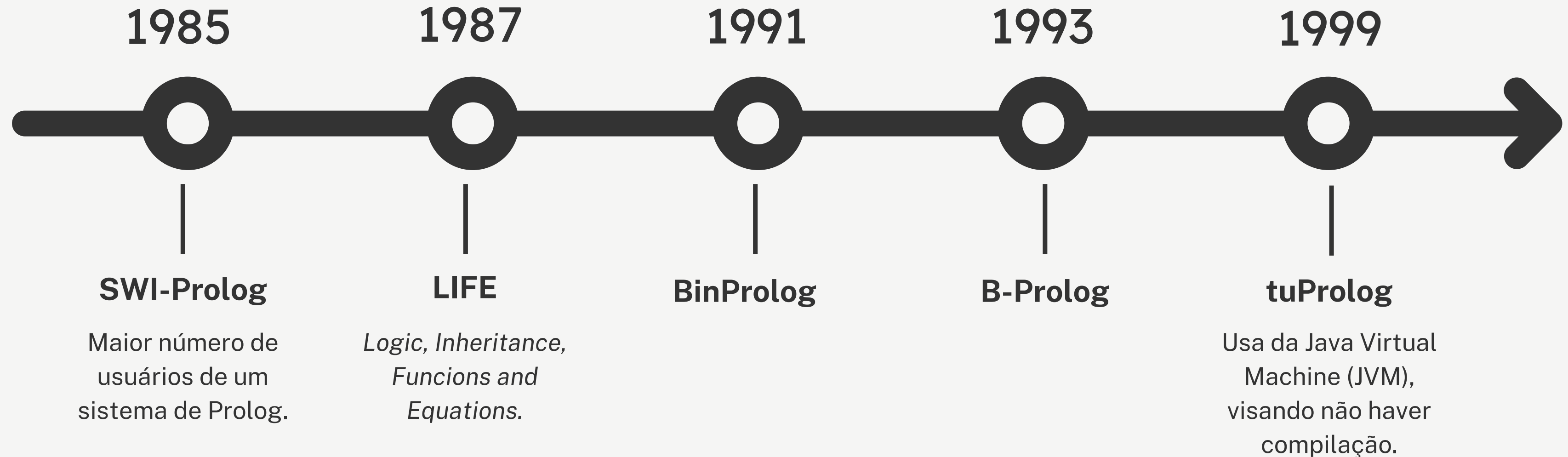
Após WAM



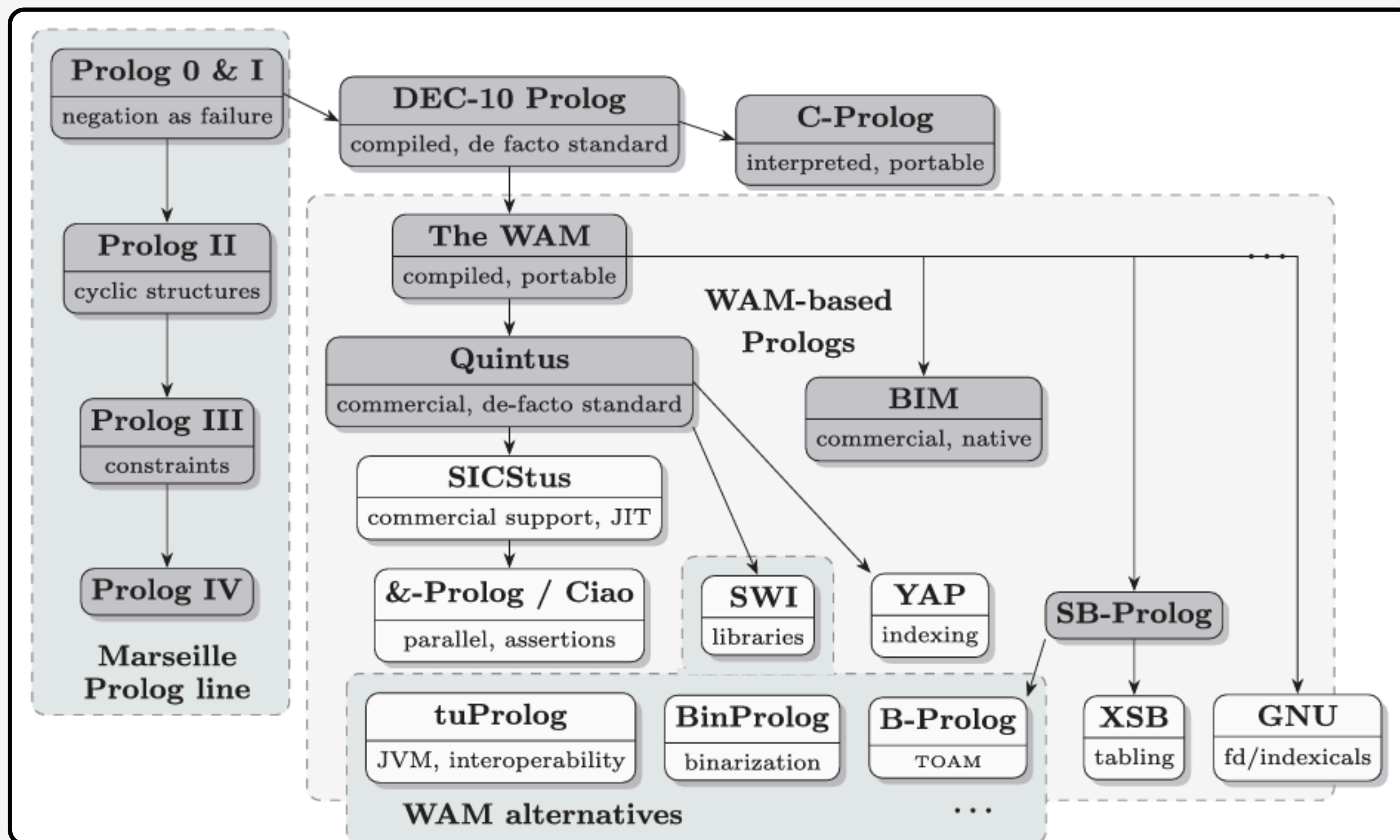
Após WAM



Alternativas ao WAM



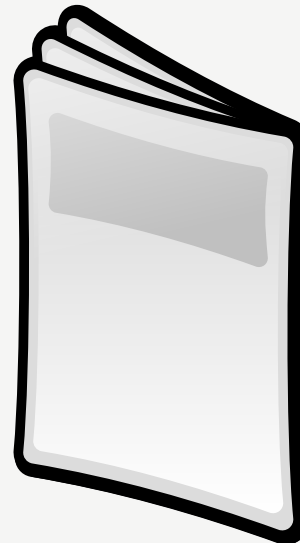
Heranças em Prolog



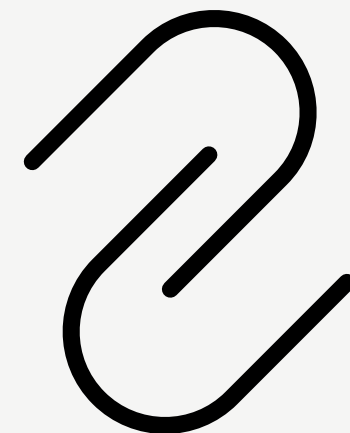
Primeiros passos da comunidade



**The First
International Logic
Programming
Conference (ICLP)**



**Journal of Logic
Programming (JLP)**



**The Association for
Logic Programming
(ALP)**



Adoção do Padrão ISO

BProlog	✓	JIProlog	✓	Poplog	✗	tuProlog	✓
Ciao	✓	JLog	✓	Scryer	✓	Visual Prolog	✗
DOS-Prolog	✗	JScripLog	✓	SICStus	✓	WIN-Prolog	✗
ECLIPSe	✓	jTrolog	✓	Strawberry	✗	XSB	✓
GNU Prolog	✓	Open Prolog	✓	SWI-Prolog	✓	YAP	✓

As distribuições do Prolog

System	Open source	Modules	Non-standard data types		Foreign language interfaces	
B-Prolog	✗	✗	arrays, sets, hashtables		C, Java	
Ciao	✓	✓	✗		C, Java, Python, JavaScript	
ECLiPSe	✓	✓	arrays, strings		C, Java, Python, PHP	
GNU Prolog	✓	✗	arrays		C, Java, PHP	
JIProlog	✓	✓	✗		Java	
SICStus	✗	✓	✗		C, Java, .NET, Tcl/Tk	
SWI	✓	✓	dicts, strings		C, C++, Java	
τ Prolog	✓	✓	✗		JavaScript	
tuProlog	✓	✗	arrays		Java, .NET, Android, iOS	
XSB	✓	✓	✗		C, Java, PERL	
YAP	✓	✓	✗		C, Python, R	

System	CLP	CHR	Tabling	Parallelism	Indexing	Type / Mode
B-Prolog	$FD, \mathcal{B}, Set$	✓	✓	✗	N-FA	✗
Ciao	$FD, \mathcal{Q}, \mathcal{R}$	✓	✓	✓	FA, MA	✓
ECLiPSe	$FD, \mathcal{Q}, \mathcal{R}, Set$	✓	✗	✓	most suitable	✗
GNU Prolog	$FD, \mathcal{B}$	✗	✗	✗	FA	✗
JIProlog	✗	✗	✗	✗	undocumented	✗
SICStus	$FD, \mathcal{B}, \mathcal{Q}, \mathcal{R}$	✓	✗	✗	FA	✗
SWI	$FD, \mathcal{B}, \mathcal{Q}, \mathcal{R}$	✓	✓	✓	MA, deep, JIT	✗
τ Prolog	✗	✗	✗	✗	undocumented	✗
tuProlog	✗	✗	✗	✓	FA	✗
XSB	$\mathcal{R}$	✓	✓	✓	all, trie	✗
YAP	$FD, \mathcal{Q}, \mathcal{R}$	✓	✓	✗	FA, MA, JIT	✗

Tipos padrões do ISO

Integer

Valores máximos e mínimos variam entre implementações

Atom

Fórmula atômica, da lógica.
Representa uma constante simbólica, representando um único conceito
Garbage Collection de atoms pode não estar disponível.

Floating point number

Uma maneira específica de representar números de ponto flutuante não é definida, mas o padrão *double* IEEE é o mais comum

Compound Term

Composto por um functor (atom que nomeia a estrutura) e argumentos. Número de argumentos é chamado de *arity*.
Arity máxima pode ser limitada e variar de acordo com implementação

Pontos principais

Características de maior compatibilidade

- Sistemas de modulo
- Programação *multithread*
- Tipagem dinâmica

Discrepâncias

- Estruturas de dados
- Estratégias de indexação
- Variáveis globais

Influência em outras linguagens lógicas

Mercury

- Linguagem lógica funcional
- Tentativa de solução para os problemas de Prolog e Haskell
- Incorpora tipagem forte, estática e polimórfica

Java

- Máquina Abstrata Java baseada na **WAM**
- Verificação de tipos em uma classe sendo fornecida por um script em Prolog

Erlang

- Baseado na *linguagem de restrição simultânea* derivada de Prolog
- Primeiro interpretador feito em Prolog

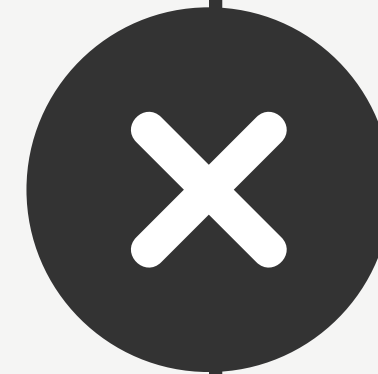
O futuro do Prolog – análise SWOT

Análise sobre os próximos passos da linguagem Prolog, refletindo sobre como ela pode evoluir diante das transformações tecnológicas atuais. Por meio de uma análise **SWOT** (Forças, Fraquezas, Oportunidades e Ameaças), a proposta é entender como Prolog pode se adaptar e continuar contribuindo, especialmente em áreas como inteligência artificial, representação de conhecimento e raciocínio automatizado.

Forças e fraquezas



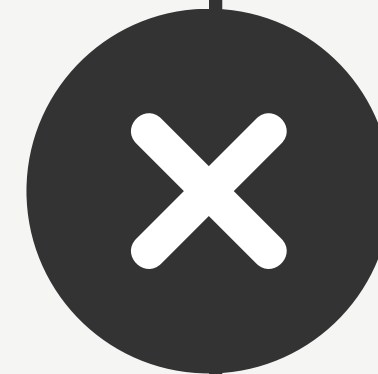
- Paradigma declarativo: descreve o “o que” em vez do “como”.
- Ideal para IA e raciocínio lógico.
- Código conciso e expressivo.



- Múltiplas implementações sem padrão unificado.
- Curva de aprendizado íngreme.
- Integração limitada com ferramentas modernas.



Prolog é forte na representação de conhecimento e raciocínio lógico, graças ao seu estilo declarativo. É eficiente em aplicações de inteligência artificial.



Prolog tem uma curva de aprendizado difícil e falta padronização entre suas implementações, o que limita a portabilidade e integração com outras tecnologias.

Oportunidades

I

**Integração com outras
linguagens:**
Aperfeiçoar interfaces com
Python e Java.

II

Novas aplicações:
Expansão para Web
Semântica, IA explicável e
robótica.

III

Apoio ao ensino:
Prolog como ferramenta
para ensinar lógica
computacional e IA.

Ameaças



Ameaças para o avanço do Prolog

Concorrência de linguagens populares

Linguagens como Python, com vasto suporte a bibliotecas de IA, são mais atraentes para desenvolvedores iniciantes.

Obsolescência

Sem evolução, Prolog corre o risco de ser substituído por linguagens mais dinâmicas e bem suportadas.

Fragmentação

A falta de um padrão único entre as implementações dificulta a portabilidade e adoção em larga escala.

Prolog: Tradição e ← Potencial Futuro

Prolog tem qualidades únicas para IA e lógica, mas precisa se atualizar para o ecossistema moderno de software. Integrar-se com outras linguagens e reforçar seu papel no ensino de lógica e IA são caminhos essenciais para garantir sua relevância futura. Essas adaptações ajudarão a manter o impacto da linguagem em novas áreas tecnológicas.

Conclusão

Primeiros passos

- Surgimento das várias implementações de Prolog disponíveis
- Divergência entre os sistemas e suas unicidades

Estado atual

- Principais sistemas Prolog
- Uso do padrão ISO
- Problemas de portabilidade

Futuro

- Pontos fortes, fracos, oportunidades e ameaças
- Comunidade
- Novas iniciativas

Obrigado

— Pela atenção!

Integrantes

Laura Menezes

Davi Puddo

Mateus Otoni

Raquel Motta

Antônio Drumond

